

GEBZE TECHNICAL UNIVERSITY

Department of Computer Science

CSE 341 — Concepts of Programming Languages

D5 — Retrospective

Part 2 Submission

22 May 2026

Halit Batuhan ASLAN

220104004003

Contents

Retrospective

Self-Assessment

Retrospective

What would you change if you were to start over?

The single change I would make most confidently is removing the semicolon terminator from the `analyze` overfitting block fields. In the current design, each field ends with `;` and fields are juxtaposed with no separator between them:

```
analyze overfitting
{
    threshold = 0.08;

    underfit_max_acc = 0.55;
}
```

This convention caused more friction than it was worth. It diverged from the model block's comma-separated style, required a dedicated parse loop, and triggered multiple rounds of corrections across the lexer, parser, and test suite every time the grammar was updated. Most importantly, the trade-off does not hold up: I justified it as a readability improvement — each field being self-terminated makes the block look clean — but in practice a user writing an `analyze` block does not benefit meaningfully from semicolons over commas. The writability cost is real and recurring; the readability gain is marginal. If I started over, I would use the same comma-separated convention as the model block, making the grammar uniform and eliminating an entire category of confusion.

What did not work and had to be scoped out?

The expression sub-language I defined in §4.3 — the full `<expr>` hierarchy from `<add_expr>` down to `<primary>` — was never semantically grounded. The §4.4 semantics section formalizes only two constructs, and the expression language has no evaluation rules at all. This was a deliberate scope decision consistent with the handout, but it left a visible gap: the type checker's R3 rule (metric literals in arithmetic) can only be triggered via a synthetic hand-crafted AST, because the parser never produces arithmetic expressions containing metric literals in the first place. In a real language implementation, the expression sub-language would need its own evaluation semantics and a proper type judgement system.

What was the single hardest design decision, and why?

The hardest decision was choosing name equivalence for the metric type. The underlying tension is real: metric and float have identical value sets ($\mathbb{R}$), identical runtime representation, and are used in the same numeric contexts inside the interpreter. Making them distinct named types requires the type checker to enforce a boundary that the grammar itself cannot enforce syntactically — `<primary>` permits metric literals in arithmetic positions, so the rejection is purely semantic. Defending this decision required assembling a three-step logical chain across two separate sections of D1 (§4.5 and §4.7), and the consistency audit in Entry #2 found that the chain was not self-contained in §4.5 alone. It was the right decision — the silent bug scenario it prevents is genuinely catastrophic in ML model selection — but it was the hardest one to formalize correctly.

Self-Assessment

Correctness — B The lexer, parser, type checker, and interpreter are all functional and consistent with the D1 specification. All 28 tests in the automated test suite pass. The main deduction is the E-ExperimentBlock formula bug found during the consistency audit — the formal rule and the prose explanation directly contradicted each other, and it required an external AI audit to surface it. The bug was fixed, but it should not have been there in the first place.

Design Justification — A Every major decision in D1 is justified with Sebesta terminology and a named trade-off. The name equivalence choice cites §6.14, the scoping choice cites §5.7 with a concrete reproducibility argument, the full evaluation choice cites §7.6 with a two-part justification, and the assignment-as-statement choice cites §7.7. §4.8 provides a paragraph per decision in my own voice with specific examples from the Ayar domain rather than generic boilerplate.

Use of Sebesta Framework — B The terminology is used accurately throughout: binding time, stack-dynamic lifetime, name equivalence, static scoping, operational semantics with big-step rules. The one imprecision found during the audit — calling not "right-associative" instead of "right-recursive" — is a genuine Sebesta §7.3 terminology error that was corrected after being caught. The expression sub-language having no formal semantics is a scope gap that lowers this from A.

Originality — A Ayar is a genuinely domain-specific language. The compare by ... higher_is_better, exclude if overfit or underfit, and select best pipeline has no equivalent in any generic scripting language. The metric type as a name-distinct primitive, the data leakage guard in E-Train, and the NoEligibleModelError loud-failure design are all domain-motivated decisions that would not appear in a general-purpose language toy project.

Code Quality — B The four components are cleanly separated: lexer, parser, type checker, and interpreter each live in their own file with no circular imports. The two-pass type checker architecture and the visitor-style interpreter dispatch are both textbook-correct. The deduction is the _last_experiment_M bug in the interpreter — a multi-experiment interaction that the initial implementation got wrong and that required a stress test to surface. Clean architecture should make such bugs visible earlier.

Exam Performance — B I can explain every design decision, every semantic rule, and every implementation choice in D1 and D2. The areas of potential weakness are the expression sub-language (which has no formal semantics and therefore thin exam coverage) and the coercion rule (which is stated in §4.5 but has no formal backing in §4.4). Both are defensible as scope decisions, but they require careful framing under exam pressure.